UNIVERSITY OF ESWATINI
Department of Computer Science
BSc Information Technology | CSC411 Integrative Programming Technologies

PATIENT PROCESSING SYSTEM

Integrated Hospital Information System

Consolidated Assignment Report

Assignment 1 - Mini Project
Course instructor: Dr M. A. Nxumalo
Report date: 8 November 2026

Group members and student numbers

Member	Student number
Vuyisa Tsabeze	202304384
Sharoon Nqobile Shoulder	202304483
Samkelo Tfwala	202301257
Nokwanele Dludlu	202302316
Luyanda Msimango	202302394

Contents

PATIENT PROCESSING SYSTEM 1
1. Introduction 3
1.1 Background and purpose 3
1.2 Scope 3
1.3 Intended outcomes 3
2. Requirement Analysis 3
2.1 Stakeholders and participating organisations 3
2.2 End-to-end process 4
2.3 Workflow branches and business rules 4
2.4 Process model and system context 5
2.5 Requirements summary 5
2.6 Modelling assumptions and items to confirm 6
3. System Design 6
3.1 Architecture overview 6
3.2 Components and communication 7
3.3 API and data exchange design 7
3.4 Data model and design patterns 7
3.5 Functional and quality requirements 8
4. Implementation Details 9
4.1 Technology stack 9
4.2 Backend implementation 9
4.3 Frontend implementation 9
4.4 Verification and current limits 9
4.5 Demonstration procedure 10
5. Conclusion 10
6. References 11

1. Introduction

1.1 Background and purpose

A hospital visit crosses multiple people, departments and, in some cases, organisations. Reception staff establish a patient and visit record; nurses assess the patient; diagnostic departments produce results; doctors and specialists interpret those results; therapists may provide referred care; and pharmacy and billing staff complete payment and dispensing. Schools, referring facilities, outside providers, medical-aid schemes and card providers can also exchange information with the hospital. The CSC411 brief identifies Patient Processing as an inter-organisational process in which clinicians make decisions using information obtained from other personnel and organisations (University of Eswatini, 2026).

The Patient Processing System coordinates information and hand-offs associated with one patient visit. A shared visit identifier connects arrival, clinical records, orders and results, referrals, therapy, prescriptions, charges, payments and dispensing. CareBridge is the user-facing name used by the prototype.

1.2 Scope

The report covers the hospital workflow from arrival and registration through assessment, in-house testing, diagnosis or referral, therapy, prescription, payment and pharmacy dispensing. The system boundary is the integrated software; staff and hospital departments are external entities to that software even though they may be inside one facility. External parties are included at the information-exchange points described by the source requirements. The prototype does not connect to live payment, medical-aid, external hospital, laboratory or school systems.

The report combines the group requirements and stakeholder analysis, process models and data-flow diagrams, system design, backend technology report and CareBridge frontend report. Design and implementation reflect an academic prototype; operational and testing limitations are stated in Section 4.

1.3 Intended outcomes

The intended outcome is one integrated, role-oriented application that preserves continuity between departments, makes a visit status visible, retains prior test results when a retest is created, consolidates hospital charges for a visit and prevents medication dispensing before payment or approval. These are assignment goals, not claims of clinical readiness or complete verification.

2. Requirement Analysis

2.1 Stakeholders and participating organisations

Stakeholders include the patient and guardian, hospital management, Reception/OPD, emergency staff, nurses, general doctors, resident and locum specialists, testing-centre staff, laboratory, radiology, cardiac services, therapy, pharmacy, billing/cashier, system administrators and external referral and payment parties (Project team, Requirements and Stakeholder Analysis, 2026).

Participant	Contribution / information exchanged
Patient / guardian	Provides identity, visit reason, referral and payment information; receives care, prescribed medicine and receipts.
Hospital / Reception and OPD	Searches or registers patients, creates visits, records visit source and routes the patient.
Emergency team and nursing	Record emergency care or nursing assessment, observations and workflow decisions.
General doctor and specialists	Review history, assessment and results; diagnose, request retests, prescribe or refer.
Testing centre, laboratory, radiology, cardiac	Route and perform only configured in-house tests; return results linked to an order and visit.

Therapy and pharmacy	Record referred therapy sessions; verify prescriptions, collect payment and dispense prescribed medication.
Cashier / billing	Record ticket or other payments, maintain bill status and issue receipts.
Schools / referring facilities	Send appointment or referral information; outcome sharing is conditional on agreement.
Outside providers	Receive referral and clinical summary; return results only where an exchange is agreed.
Medical-aid scheme / bank-card provider	May verify membership or authorise payment; prototype approvals are simulated.
System administrator	Manages staff accounts, roles, fee catalogues and audit review.

2.2 End-to-end process

The process is organized into seven logical functions. The patient and visit are the common records that connect each function; the DFD stores are logical groupings, not a requirement for exactly four physical database tables.

Process	Responsibility
1.0 Arrival and registration	Capture arrival/source; find or register the patient; select emergency, medication-only or standard routing.
2.0 Emergency care and deferred charges	Create a temporary emergency visit, record staff and care, complete details after stabilisation, and carry eligible charges forward.
3.0 Visit, ticket fee and nurse assessment	For standard non-emergency visits, record ticket payment before assessment; capture the nurse assessment and route the case.
4.0 In-house tests and results	Create and route orders, record results, and preserve earlier orders/results when a retest is created.
5.0 Diagnosis and referrals	Review clinical information; diagnose, request more tests, refer to a resident/locum specialist, or refer outside if undiagnosed.
6.0 Prescriptions and pharmacy dispensing	Retrieve and verify a linked prescription, confirm payment/approval, dispense and record the event.
7.0 Billing and payment	Maintain one bill per visit, record cash/card/medical-aid payment and issue a receipt.

2.3 Workflow branches and business rules

At arrival, an emergency patient is treated before full registration or payment, with a temporary record and deferred charges. Non-emergency patients are searched or registered at Reception/OPD. A school appointment or referral continues through the standard ticket and nurse stages. A medication-collection-only visit requires an existing prescription and skips the ticket and nurse stages; the medication charge is still paid/approved before dispensing (the project source identifies this as a working assumption).

For a standard visit, the ticket fee is paid before nurse assessment. The requirements document specifies two nurse outcomes: minor illness treatment using low-schedule drugs only, or an in-house test request. The in-house test set is blood, glucose, urine, pregnancy, pap smear, X-ray and ECG. A general doctor may request a new retest order on the same visit, diagnose and prescribe, or refer to a resident/locum specialist. A specialist who cannot diagnose may refer outside. A doctor or specialist may refer a diagnosed patient to in-house therapy. At the payment point, one visit bill brings together applicable ticket, test, medication, therapy and deferred hospital charges; medicine is dispensed only when prescribed and after payment or medical-aid approval.

Payment may be cash, card or medical aid. Card and medical-aid authorisation is simulated; no real provider integration is claimed. External-provider test charges and deferred charges created by the host hospital are not necessarily the same item. The process-modelling document records this as an issue for confirmation; this report does not assign an invented fee or resolve who collects an outside provider's charge.

2.4 Process model and system context

The context model treats the integrated software as one process and identifies 18 separately named external entities. The process model then decomposes the system into the seven major functions above, with logical stores for patient/visit/clinical records (D1), test orders/results (D2), prescriptions/dispensing (D3), and bills/payments (D4). External entities exchange information through system processes; they are not shown as directly accessing data stores (Project team, Process Modelling and DFD, revised 8 November 2026).

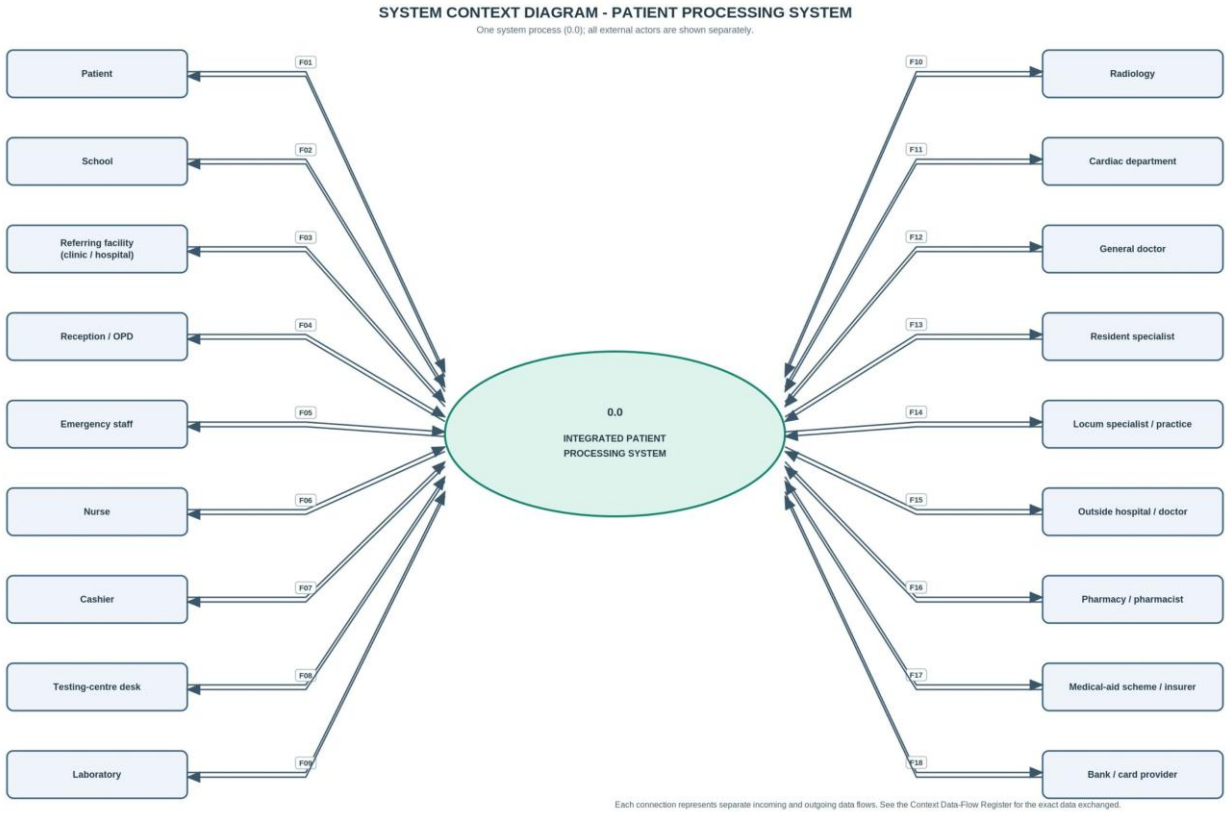


Figure 1. System context diagram supplied with the process-modelling contribution (18 external entities).

2.5 Requirements summary

The detailed functional requirements are numbered FR-01 onward in the requirements contribution. The core Must requirements are grouped here; the source requirement register remains the detailed traceability baseline.

Requirement group	Core system obligations
Arrival / registration (FR-01 to FR-04)	Record arrivals; search/register patients; assign a unique ID and warn about possible duplicates; distinguish medication-only visits.
Emergency (FR-06 to FR-08, FR-10)	Allow treat-first temporary records, record emergency clinicians, capture later details, support prescriptions and defer eligible payment.
Billing (FR-12 to FR-15)	Ticket fee before standard assessment; one bill per visit; cash/card/medical-aid recording; pharmacy settlement of deferred hospital charges.
Nursing (FR-18 to FR-20)	Record complaint/vitals/notes; capture the decision; constrain minor-illness treatment to low-schedule medicines.

Testing (FR-21 to FR-26)	Route only the seven in-house tests; record and consolidate results; create retests without overwriting history; add charges.
Diagnosis/referral (FR-27 to FR-36)	Show clinical history/results; record diagnosis and treatment plan; support retests, specialist/outside referral, in-house therapy and charges.
Pharmacy (FR-37 to FR-40)	Create/retrieve prescriptions; support medication-only route; dispense prescribed items only after payment/approval.
Integration/access (FR-42 to FR-44)	Provide JSON REST-style APIs, login/role checks and an audit trail.

2.6 Modelling assumptions and items to confirm

Outside charges: the requirements state patients pay for outside tests, while another rule defers amounts to the host pharmacy. The sources do not clearly distinguish third-party charges from host-hospital charges. No price or collection rule is invented; confirm the intended accounting boundary.

Medication collection: the source flow skips ticket/nurse but requires payment before dispensing. Application documentation assumes use of an active undispensed prescription from an earlier visit. Treat this as a working assumption, not a confirmed rule.

Fees and schedules: authoritative amounts and the low-schedule catalogue were not supplied. The prototype uses labelled demo catalogue values; these are not real fees or clinical guidance.

3. System Design

3.1 Architecture overview

The design uses a three-tier client-server architecture implemented as a modular monolith. Department-specific interfaces share one PHP application and one relational database. A single application was selected rather than microservices because departments operate on the same patient and visit records, and one transaction boundary keeps related changes simpler for an academic local deployment (Project team, System Design and Architecture, 2026).

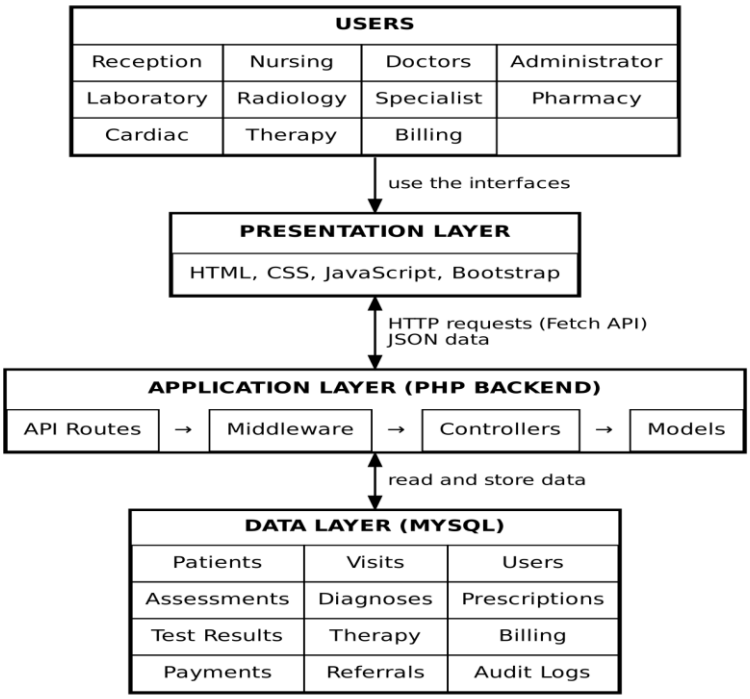


Figure 2. Three-tier architecture from the design contribution: department users, browser presentation, PHP application and shared MySQL data.

3.2 Components and communication

Layer / component	Responsibility
Presentation	Department browser pages, forms, queues and status views. HTML/CSS provide structure; JavaScript and Fetch send requests and display responses.
API routes	Map HTTP methods and resource paths to application operations through the central backend entry point.
Middleware	Establishes staff session and checks role before protected operations.
Controllers	Validate request data, apply workflow rules and prepare JSON responses.
Models / data access	Use PDO and parameterized queries to retrieve or persist relational data. Transactions protect selected multi-record actions.
MySQL/MariaDB	Persists patients, visits, clinical records, tests, referrals, prescriptions, inventory, bills, payments, users, roles and audit events.
Audit support	Records selected user actions with patient/visit context and timestamp.

A typical request starts in a department page, is sent over HTTP with JSON, passes through the route and session/role middleware, is handled by a controller and data-access code, and returns JSON with an HTTP status. For example, patient registration returns a created patient identifier; an invalid or unauthorised request returns an error response.

3.3 API and data exchange design

The API is REST-style: paths identify resources, HTTP methods express retrieval or change, JSON represents request and response data, and status codes communicate the outcome. The browser uses PHP sessions for authentication, so this is not a fully stateless REST deployment. Exact implemented paths remain provisional and differ from Person 3's proposed paths; the mapping is documented in docs/API_MAPPING.md.

Method / status	Intended use
GET	Retrieve a patient, visit, queue, test order, bill, receipt or report.
POST	Create or submit a visit, assessment, consultation, order result, referral, payment or dispensing action.
PATCH	Update selected patient details, visit status or other permitted fields.
200 / 201	Successful retrieval/update / resource creation.
400 / 401 / 403 / 404	Malformed request / unauthenticated / forbidden / resource missing.
409 / 422 / 500	Workflow conflict / invalid data / unexpected server failure.

Representative resources include patients, visits, nursing assessments, consultations and diagnoses, test orders and results, specialist referrals/outcomes, therapy referrals and sessions, prescriptions, dispensing, bills/payments, authentication, user administration and audit records. Current examples include POST /api/auth/login, POST /api/reception/visits, POST /api/nursing/visits/{id}/assessment, POST /api/consultations, POST /api/laboratory/results and POST /api/pharmacy/visits/{id}/payments. Endpoint names and request contracts are provisional pending final team reconciliation.

3.4 Data model and design patterns

The conceptual model centres on Patient and Visit. One patient may have multiple visits; each visit can be linked to assessments, consultations/diagnoses, test orders/results, referrals, therapy records, prescriptions, a bill and payments. User and Role support access control; AuditLog records selected actions. A retest is a new TestOrder linked to the same visit, preserving earlier results. Billing uses one visit bill with line items and one or more payments. These relationships support D1-D4 logical data groupings in Section 2.

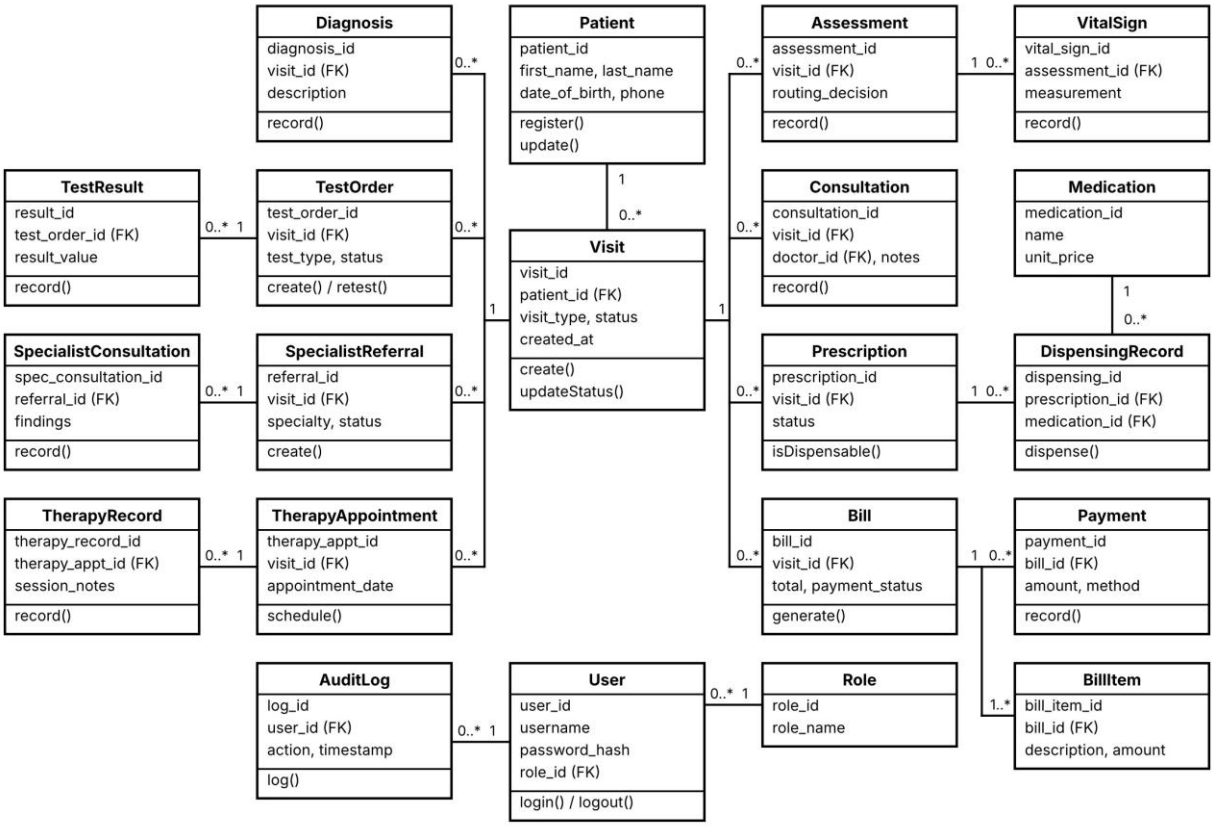


Figure 3. Conceptual class diagram from the design contribution. It is a design artefact, not a claim that every class or relationship is validated against the running schema.

The source UML includes test-type examples beyond the seven allowed in the requirements. The report and intended implementation scope is restricted to blood, glucose, urine, pregnancy, pap smear, X-ray and ECG; extra UML test examples are excluded. The main patterns are layered architecture, MVC-style separation, route middleware for authentication/roles and reusable data-access functions. These choices keep responsibilities understandable without a framework or separate services.

3.5 Functional and quality requirements

Quality attribute	Design target / rationale
Security and access	Authenticate staff; enforce permissions server-side; hash passwords; validate input; use parameterized database queries; log selected actions.
Data integrity	Use relational keys and transactions where related clinical, billing or stock records change together. Preserve history rather than overwriting results.
Interoperability	Use HTTP methods and JSON so a compatible client can call the API. No live third-party integrations are claimed.
Usability	Provide role-oriented department pages, visible workflow states, and clear success/error feedback.
Maintainability	Separate route, middleware, controller, model and department responsibilities within the simple PHP application.
Performance	The design contribution sets about two seconds for common actions as a target under expected load; this target has not been measured.

Reliability and recovery	Saved operations should persist consistently; backup/recovery is a design need, not a verified service in this local prototype.

Role-based access is enforced by the backend, not merely by hiding navigation. Roles correspond to operational responsibilities such as receptionist, nurse, doctor, testing staff, specialist, therapist, pharmacist, cashier and administrator. Emergency, locum and medical-aid pathways remain subject to the implementation and test limits below.

4. Implementation Details

4.1 Technology stack

Technology	Use in the prototype
PHP	Backend controllers, API routes, sessions, middleware and server-rendered PHP pages.
MySQL / MariaDB with PDO	Relational persistence and prepared statements for database queries.
HTML, CSS, JavaScript	Department interfaces, layouts, navigation and browser interaction.
JSON over HTTP	Request/response format between frontend and backend API.
Apache / XAMPP	Local Windows web and database development environment.
VS Code, Git/GitHub, Postman	Editing, version control/collaboration and API request testing.

The stack is suitable for the local demo environment described in project documentation. It avoids a separate application server or ORM and is intended to run under XAMPP with a configurable MySQL-compatible database (Project team, Backend Report, 2026; CareBridge Frontend Development Report, 2026).

4.2 Backend implementation

The backend has a central entry point at backend/index.php and route definitions under backend/routes/. Authentication, administration, reception, billing, nursing, general doctor, testing desk, laboratory, radiology, cardiac, specialist referrals, therapy, pharmacy, visit tracking, audit viewing and reports are represented in the current repository. API route details and proposed-path differences are tracked in docs/API_MAPPING.md.

The backend uses PHP sessions and role checks. Password verification uses PHP password hashing functions; database access uses PDO and parameterized statements. Current workflow documentation describes transactions for selected related operations, including payment, test-result billing and stock/dispensing changes. Card and medical-aid approvals are simulated, and sample fee and medicine catalogues contain demo values.

4.3 Frontend implementation

The CareBridge frontend is divided into role-oriented department workspaces with a shared visual style and navigation. The repository includes areas for administration, reception, nursing, general doctor, testing desk, laboratory, radiology, cardiac, billing, specialists, therapy and pharmacy, plus visit status. Pages generally separate queues, in-progress work, details, history and completion. The dashboard contains placeholder statistics and activity information; these are not live operational figures.

4.4 Verification and current limits

Repository documentation reports that PHP syntax checks and an HTTP health check passed on 7 November 2026. Those checks predate the emergency-nursing changes and do not verify a user-facing workflow. The documented ten demo scenarios are test instructions, not observed test results. As of the current demo record, none of the complete authenticated scenarios has been recorded as passing because role accounts/passwords were unavailable; emergency nursing and cardiac flows still require migrations and authenticated end-to-end verification (docs/DEMO.md).

Area	Implementation status in project documentation
Core modules	Pages/API modules exist for administration, reception, billing, nursing, general doctor, testing, laboratory, radiology, cardiac, specialists, therapy, pharmacy, visit status, audit viewing and reports. Presence is not end-to-end proof.
Emergency	Emergency nursing pages/API and doctor handover are described as implemented on 8 November; required migration and authenticated workflow test remain outstanding.
Medication-only	Skips ticket and nurse path; documentation assumes an active undispensed prescription from an earlier visit; payment before dispensing still applies.
Payment providers	Card and medical-aid responses are simulated; no real bank/insurer integration.
Area	**Implementation status in project documentation**
Fees and medicines	Sample/demo catalogue values only; not real prices or medical guidance.
API specification	Current routes are provisional and differ from the design contribution; final reconciliation is pending.
End-to-end tests	Role-based scenarios S1-S10 are not recorded as passed; demo role accounts must be created locally.
Production suitability	Not a production or clinical system; fictional/demo data only.

4.5 Demonstration procedure

For a local demonstration, start Apache and MySQL in XAMPP, import database/hosp.sql and apply migrations required by the chosen module, configure database access, create staff roles/accounts using locally chosen passwords, and use fictional patient records. Follow docs/DEMO.md to demonstrate a selected workflow. Use Postman with the session cookie for API checks. Describe only the specific path actually run; do not generalize it to all modules or scenarios.

Before submission, the team should insert each member's full name and student number on the cover, agree the nurse-routing interpretation and outside-charge boundary, reconcile API paths, run intended demonstration paths with role accounts, and update the test table with observed results. These are remaining actions, not assertions that they have been completed.

5. Conclusion

The Patient Processing System models a shared hospital visit across reception, clinical assessment, diagnostic testing, diagnosis/referral, therapy, pharmacy and billing. The requirements establish critical rules for emergency treatment, ticket payment, medication-only visits, the seven in-house tests, retests, specialist/outside referrals, deferred hospital charges and paid-before-dispensing. The design organizes these activities as a modular PHP application with a browser interface, REST-style JSON API and shared relational database. The repository provides department pages and backend modules for much of this scope, but available documentation does not demonstrate that every workflow passed an end-to-end test.

The consolidated result is an academic prototype report, not a claim of a fully verified hospital system. The most important completion items are to resolve nurse-route and outside-charge differences in the source documents, add actual team-member details, reconcile proposed and implemented API contracts, and record authenticated demonstration results using fictional data. Simulated approvals, demo fees and incomplete end-to-end verification remain explicit limitations.

6. References

Bray, T. (Ed.) (2017). *The JavaScript Object Notation (JSON) Data Interchange Format* (RFC 8259). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc8259

Clegg, D. and Barker, R. (1994). *Case Method Fast-Track: A RAD Approach*. Addison-Wesley.

de Carvalho Junior, M. A. and Bandiera-Paiva, P. (2018). Health information system role-based access control: Current security trends and challenges. *Journal of Healthcare Engineering*, 2018, Article 6510249. https://doi.org/10.1155/2018/6510249

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine. https://ics.uci.edu/~fielding/pubs/dissertation/abstract.htm

Iraq Ministry of Health and USAID Primary Health Care Project in Iraq (2012). *The Referral System Revised*. University Research Co., LLC. https://pdf.usaid.gov/pdf_docs/PA00KCB7.pdf

International Organization for Standardization (2023). *ISO/IEC 25010:2023 Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - Product quality model*. https://www.iso.org/standard/78176.html

ISO/IEC/IEEE (2018). *ISO/IEC/IEEE 29148:2018 Systems and software engineering - Life cycle processes - Requirements engineering*. 2nd ed. ISO/IEC/IEEE.

Kingdom of Eswatini (2022). *Data Protection Act, 2022 (Act No. 5 of 2022)*. https://www.esccom.org.sz/legislation/DATA%20PROTECTION%20ACT.pdf

MEASURE Evaluation (2013). *Referral Systems Assessment and Monitoring Toolkit*. Report MS-13-60. University of North Carolina at Chapel Hill. https://measureevaluation.cpc.unc.edu/resources/publications/ms-13-60.html

Mendelow, A. L. (1981). Environmental scanning - the impact of the stakeholder concept. *ICIS 1981 Proceedings*, Paper 20. https://aisel.aisnet.org/icis1981/20

Project team (2026). *Person 1: Requirements and Stakeholder Analysis*. CSC411 mini-project contribution, revised source supplied for consolidation.

Project team (2026). *Person 2: Patient Processing System Process Modelling and Data Flow Diagrams*. CSC411 mini-project contribution, revised 8 November 2026.

Project team (2026). *Person 3: System Design and Architecture*. CSC411 mini-project contribution, updated source supplied for consolidation.

Project team (2026). *Backend Report: Backend Technologies and Their Purposes*. CSC411 mini-project contribution.

Project team (2026). *CareBridge Frontend Development Report*. CSC411 mini-project contribution.

Project team (2026). *Hospital Integrated Patient Processing System repository documentation*, including README.md, docs/DEMO.md, docs/API_MAPPING.md and module guides. Accessed 8 November 2026.

Sommerville, I. (2016). *Software Engineering*. 10th ed., global ed. Harlow: Pearson.

University of Eswatini, Department of Computer Science (2026). *CSC411 Integrative Programming Technologies: Assignment 1 Mini Project*. Course brief issued 24 May 2026; due 4 November 2026. Unpublished course document.